

P5: Structuring Data

Name: Alston Liow Guan You

Student ID: 37440985

Date: 06 August 2026

Overview

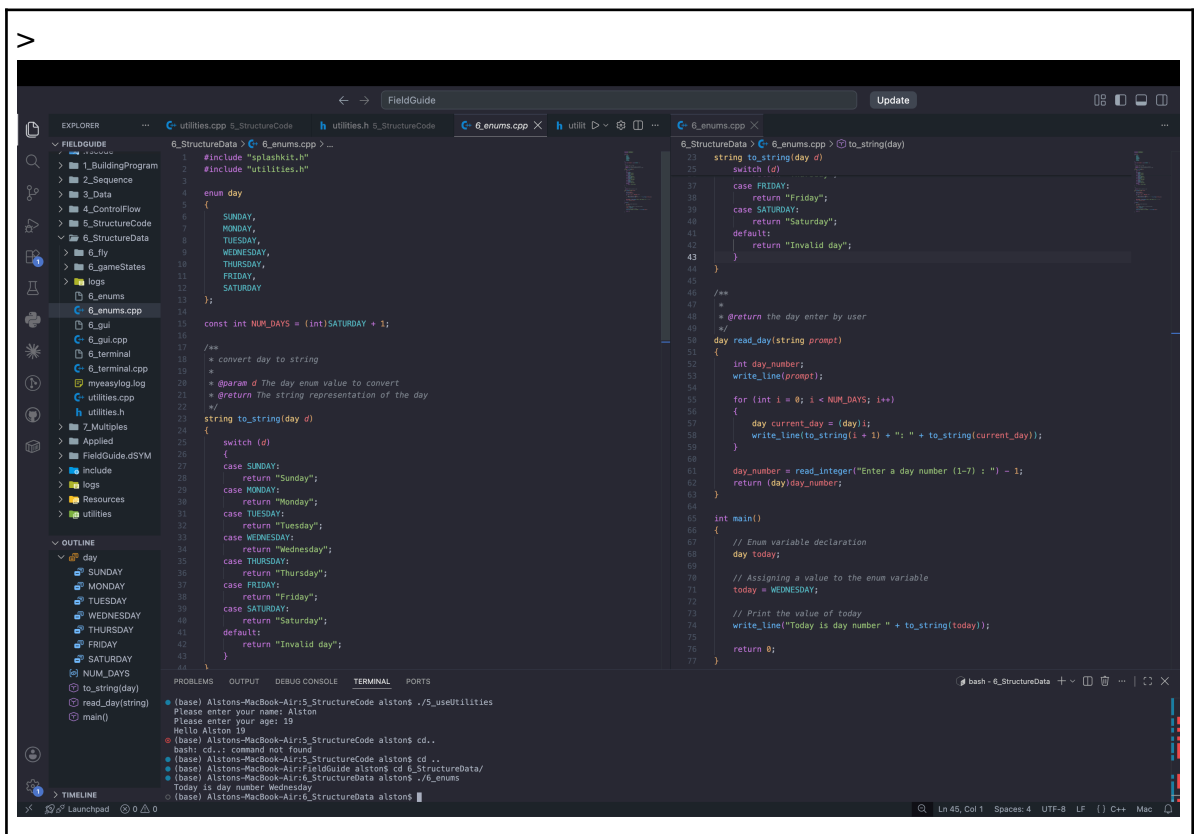
This task is an opportunity to explore and use language mechanisms to organise data within your programs.

Tip: Abstract thinking about data

As with functions and procedures, you are now dealing with building abstract tools. Designing your own data types will help you organise the kinds of things you can store in variables and parameters, and return from functions.

Do the following:

- Everything you need is in the [Structuring Data](#) chapter of our [Programmer's Field Guide](#). The [Intro Tours](#) walk through the concepts, and should give you a good start.
- Add a screenshot showing you were able to create and use enumerations. You can use the [explore enums tour](#).



```

1 #include "utilities.h"
2 #include "utilities.h"
3
4 enum day
5 {
6     SUNDAY,
7     MONDAY,
8     TUESDAY,
9     WEDNESDAY,
10    THURSDAY,
11    FRIDAY,
12    SATURDAY
13 };
14
15 const int NUM_DAYS = (int)SATURDAY + 1;
16
17 /**
18  * Convert day to string
19  *
20  * @param d The day enum value to convert
21  * @return The string representation of the day
22  */
23 string to_string(day d)
24 {
25     switch (d)
26     {
27         case SUNDAY:
28             return "Sunday";
29         case MONDAY:
30             return "Monday";
31         case TUESDAY:
32             return "Tuesday";
33         case WEDNESDAY:
34             return "Wednesday";
35         case THURSDAY:
36             return "Thursday";
37         case FRIDAY:
38             return "Friday";
39         case SATURDAY:
40             return "Saturday";
41         default:
42             return "Invalid day";
43     }
44 }
45
46 /**
47  * Return the day enter by user
48  */
49 day read_day(string prompt)
50 {
51     int day_number;
52     write_line(prompt);
53     for (int i = 0; i < NUM_DAYS; i++)
54     {
55         day current_day = (day)i;
56         write_line(to_string(i + 1) + ": " + to_string(current_day));
57     }
58     day_number = read_integer("Enter a day number (1-7) : ");
59     return (day)day_number;
60 }
61
62 int main()
63 {
64     // Enum variable declaration
65     day today;
66
67     // Assigning a value to the enum variable
68     today = WEDNESDAY;
69
70     // Print the value of today
71     write_line("Today is day number " + to_string(today));
72
73     return 0;
74 }

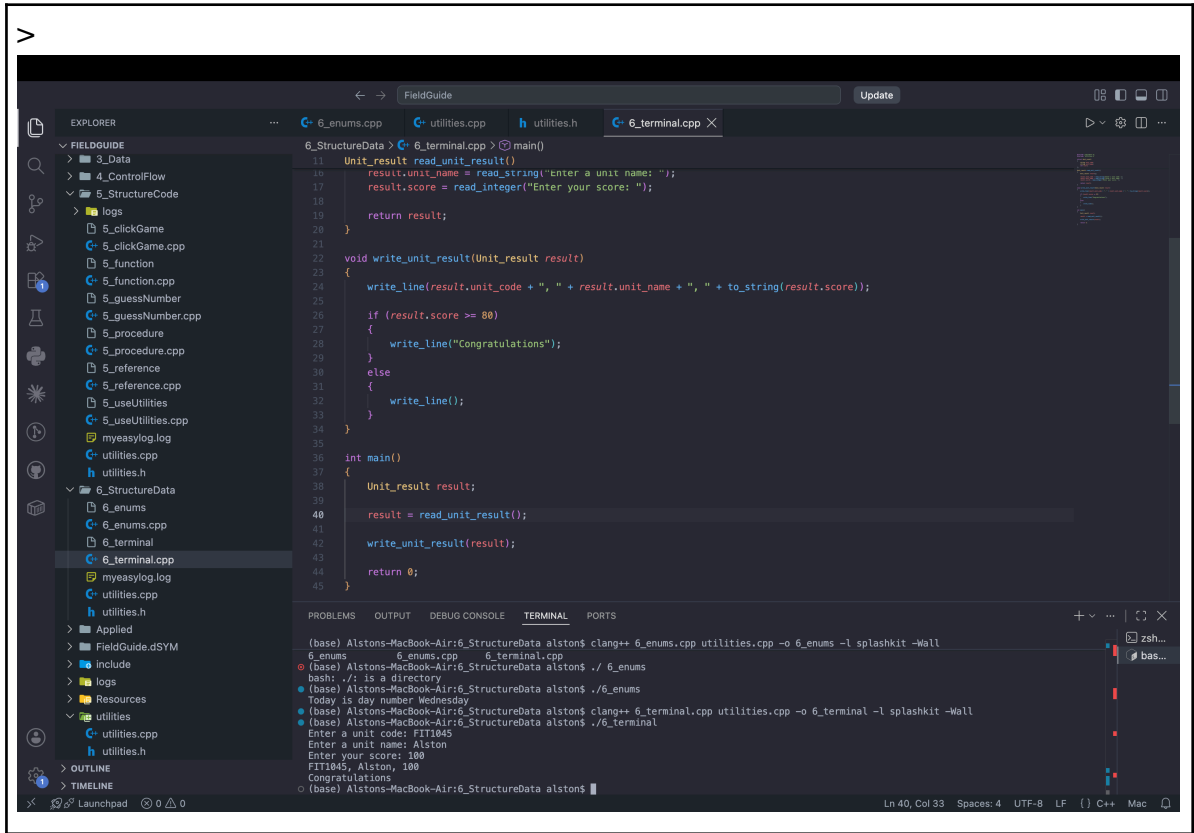
```

```

(base) Alstons-MacBook-Air:5_StructureCode alstons ./5_useUtilities
Please enter your name: Alston
Please enter your age: 19
Hello Alston 19
(base) Alstons-MacBook-Air:5_StructureCode alstons cd ..
bash: cd: command not found
(base) Alstons-MacBook-Air:5_StructureCode alstons cd 6_StructureData/
(base) Alstons-MacBook-Air:6_StructureData alstons ./6_enum
Today is day number Wednesday
(base) Alstons-MacBook-Air:6_StructureData alstons

```

- Add a screenshot showing you were able to create and use structs. You can use the [explore structs terminal tour](#).

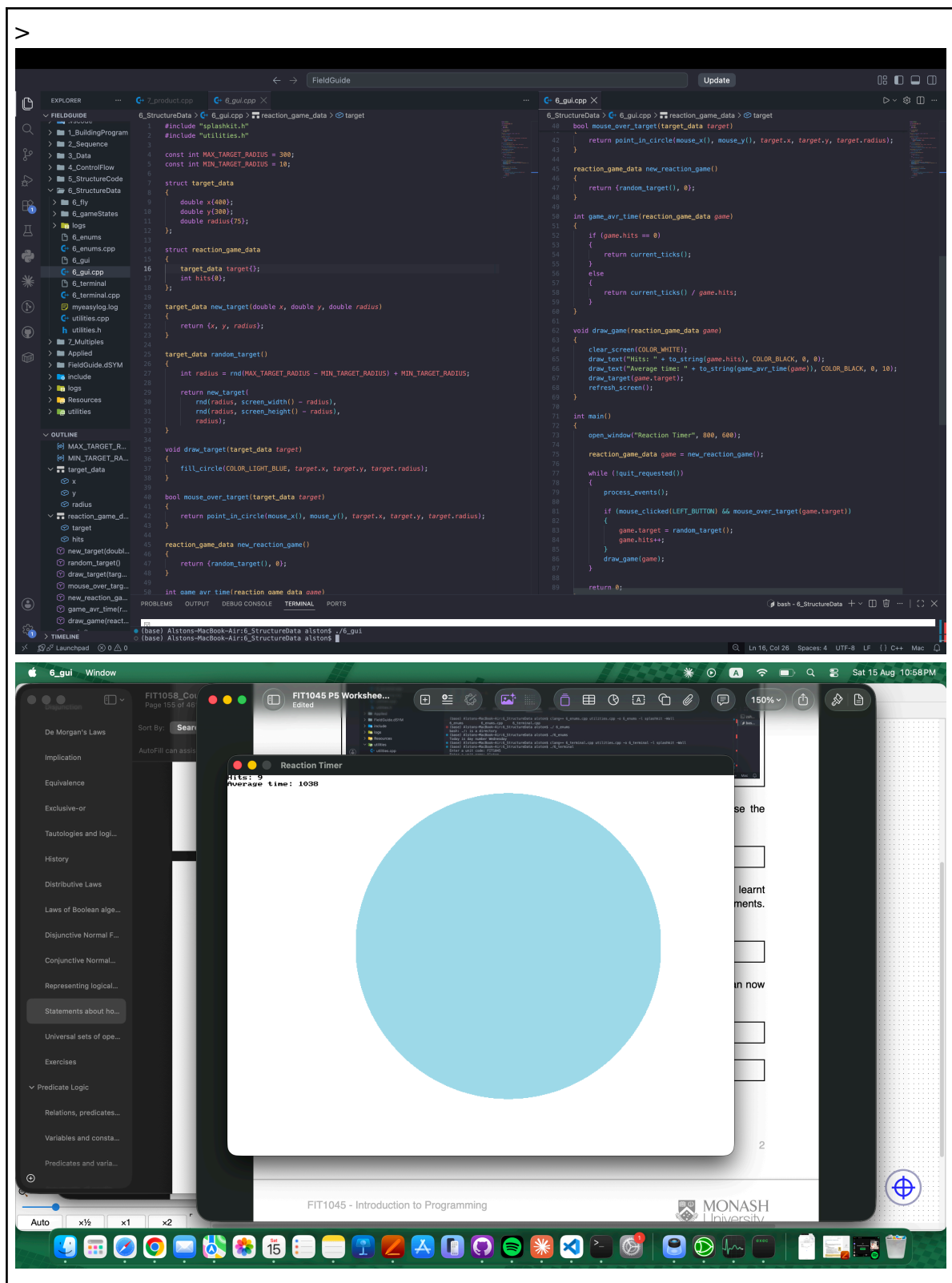


```

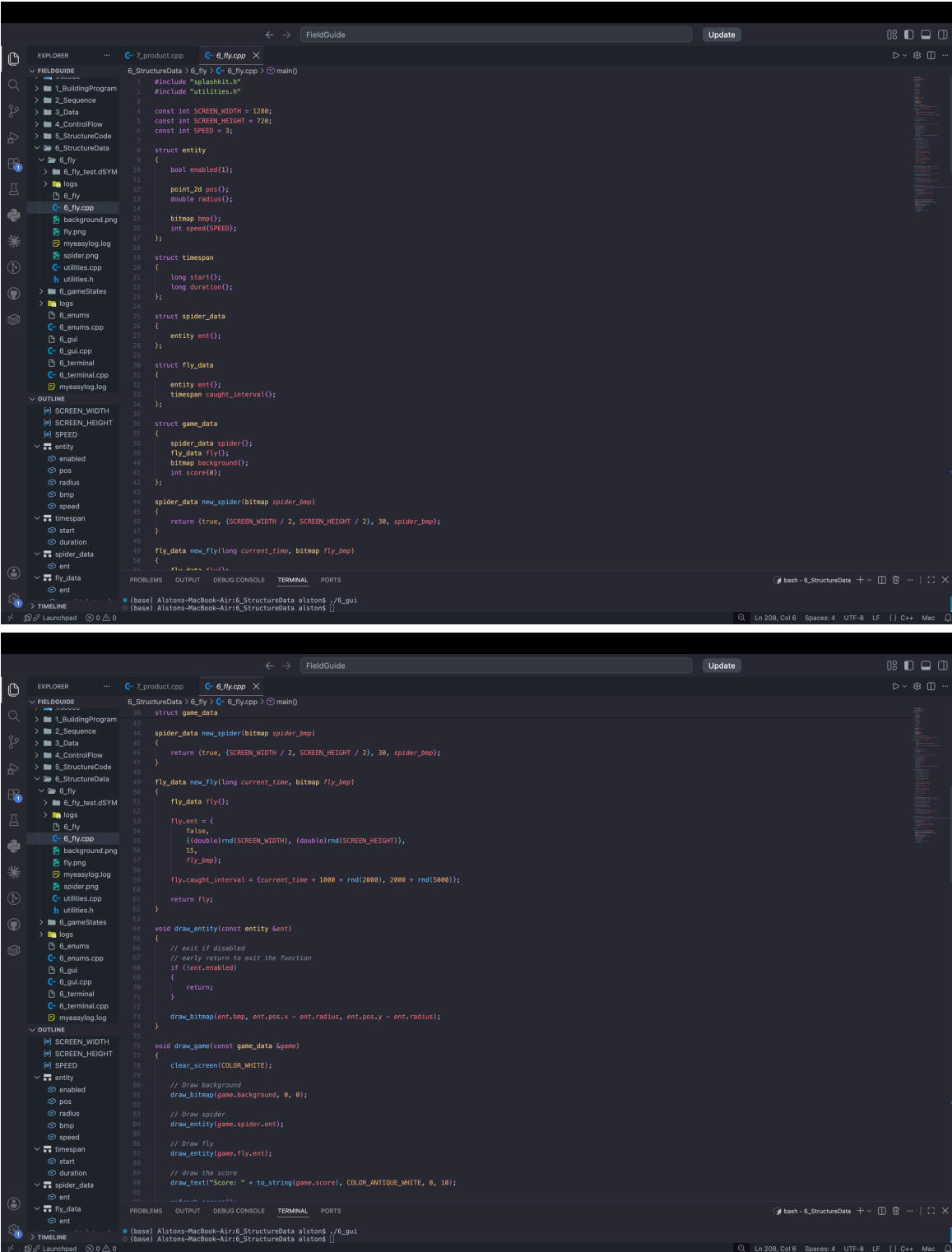
6_StructureData > C++ 6_terminal.cpp > main()
11 Unit_result read_unit_result()
12 {
13     result.unit_name = read_string("Enter a unit name: ");
14     result.score = read_integer("Enter your score: ");
15
16     return result;
17 }
18
19 void write_unit_result(Unit_result result)
20 {
21     write_line(result.unit_code + " " + result.unit_name + " " + to_string(result.score));
22
23     if (result.score >= 80)
24     {
25         write_line("Congratulations");
26     }
27     else
28     {
29         write_line();
30     }
31 }
32
33 int main()
34 {
35     Unit_result result;
36
37     result = read_unit_result();
38
39     write_unit_result(result);
40
41     return 0;
42 }
43
44
45
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
(base) Alstons-MacBook-Air:6_StructureData alston$ clang++ 6_enums.cpp utilities.cpp -o 6_enums -l splashkit -Wall
6_enums
(base) Alstons-MacBook-Air:6_StructureData alston$ ./ 6_enums
bash: ./ : is a directory
(base) Alstons-MacBook-Air:6_StructureData alston$ ./ 6_enums
Today is day number: Wednesday
(base) Alstons-MacBook-Air:6_StructureData alston$ clang++ 6_terminal.cpp utilities.cpp -o 6_terminal -l splashkit -Wall
6_terminal
(base) Alstons-MacBook-Air:6_StructureData alston$ ./ 6_terminal
Enter a unit code: FIT1045
Enter a unit name: Alston
Enter your score: 100
FIT1045, Alston, 100
Congratulations
(base) Alstons-MacBook-Air:6_StructureData alston$

```

- Add a screenshot showing you were able to create and use structs. You can use the [explore structs GUI tour](#).



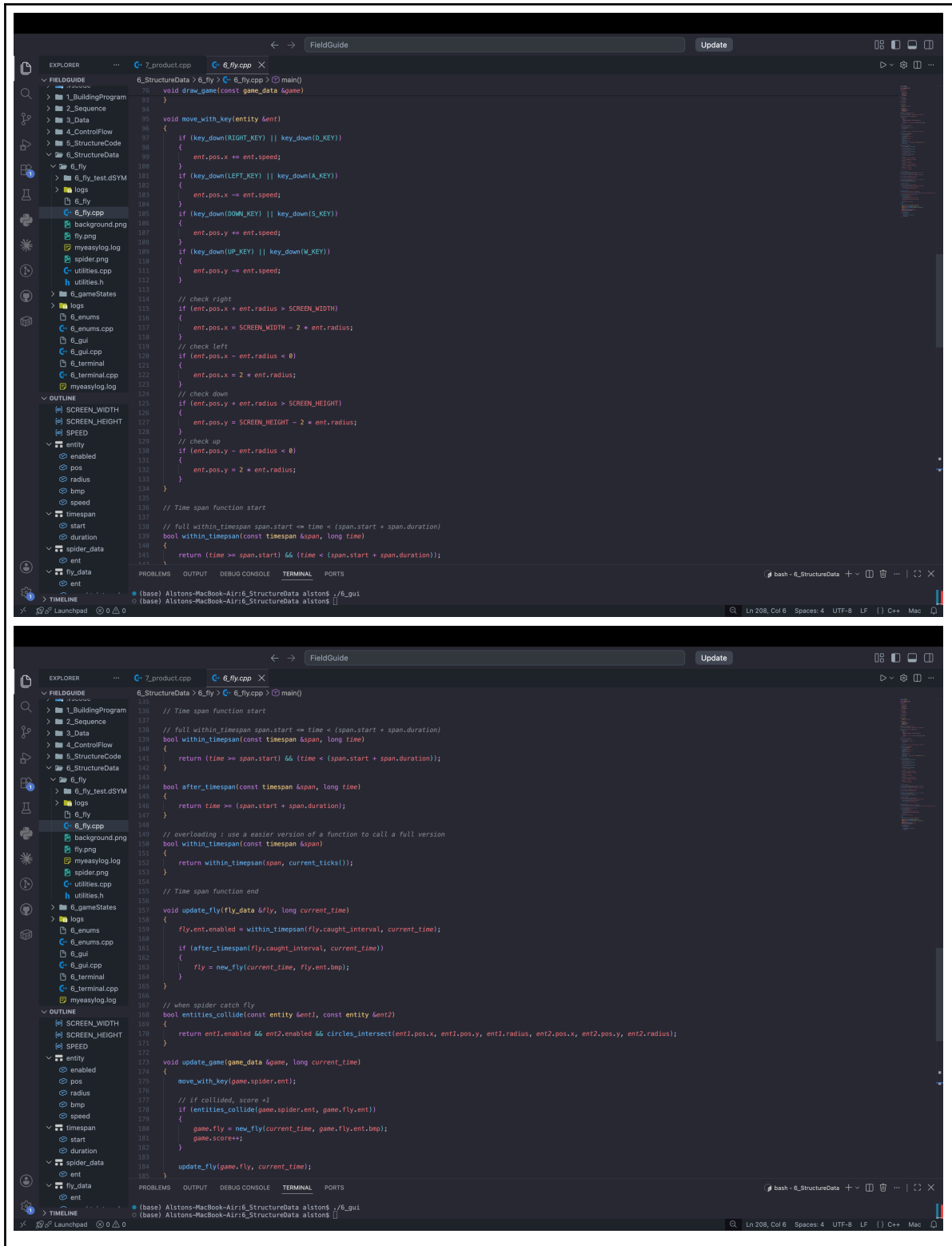
- If you want to extend your capabilities further, try putting everything you have learnt together to create a small program. Then add a screenshot showing your achievements. You can use the [fly catch tour](#).



```

1  #include "splashkit.h"
2  #include "utilities.h"
3
4  const int SCREEN_WIDTH = 1280;
5  const int SCREEN_HEIGHT = 720;
6  const int SPEED = 5;
7
8  struct entity
9  {
10     bool enabled();
11
12     point_2d pos();
13     double radius();
14
15     bitmap bmp();
16     int speed(SPEED);
17 };
18
19 struct timespan
20 {
21     long start();
22     long duration();
23 };
24
25 struct spider_data
26 {
27     entity ent();
28 };
29
30 struct fly_data
31 {
32     entity ent();
33     timespan caught_interval();
34 };
35
36 struct game_data
37 {
38     spider_data spider();
39     fly_data fly();
40     bitmap background();
41     int score();
42 };
43
44 spider_data new_spider(bitmap spider_bmp)
45 {
46     return (true, (SCREEN_WIDTH / 2, SCREEN_HEIGHT / 2), 30, spider_bmp);
47 }
48
49 fly_data new_fly(long current_time, bitmap fly_bmp)
50 {
51     fly_data fly();
52
53     fly.ent = {
54         false,
55         ((double)rnd(SCREEN_WIDTH), (double)rnd(SCREEN_HEIGHT)),
56         15,
57         fly_bmp;
58     };
59     fly.caught_interval = {current_time + 1000 + rnd(2000), 2000 + rnd(5000)};
60     return fly;
61 }
62
63 void draw_entity(const entity &ent)
64 {
65     // exit if disabled
66     // early return to exit the function
67     if (!ent.enabled)
68     {
69         return;
70     }
71
72     draw_bitmap(ent.bmp, ent.pos.x - ent.radius, ent.pos.y - ent.radius);
73 }
74
75 void draw_game(const game_data &game)
76 {
77     clear_screen(COLOR_WHITE);
78
79     // Draw background
80     draw_bitmap(game.background, 0, 0);
81
82     // Draw spider
83     draw_entity(game.spider.ent);
84
85     // Draw fly
86     draw_entity(game.fly.ent);
87
88     // draw the score
89     draw_text("Score: " + to_string(game.score), COLOR_ANTIQUE_WHITE, 0, 10);
90 }

```



```

6_StructureData > 6_fly > 6_fly.cpp > @main()
void draw_game(const game_data &game)
{
    // draw the fly
    if (key_down(RIGHT_KEY) || key_down(D_KEY))
    {
        ent.pos.x += ent.speed;
    }
    if (key_down(LEFT_KEY) || key_down(A_KEY))
    {
        ent.pos.x -= ent.speed;
    }
    if (key_down(DOWN_KEY) || key_down(S_KEY))
    {
        ent.pos.y += ent.speed;
    }
    if (key_down(UP_KEY) || key_down(W_KEY))
    {
        ent.pos.y -= ent.speed;
    }

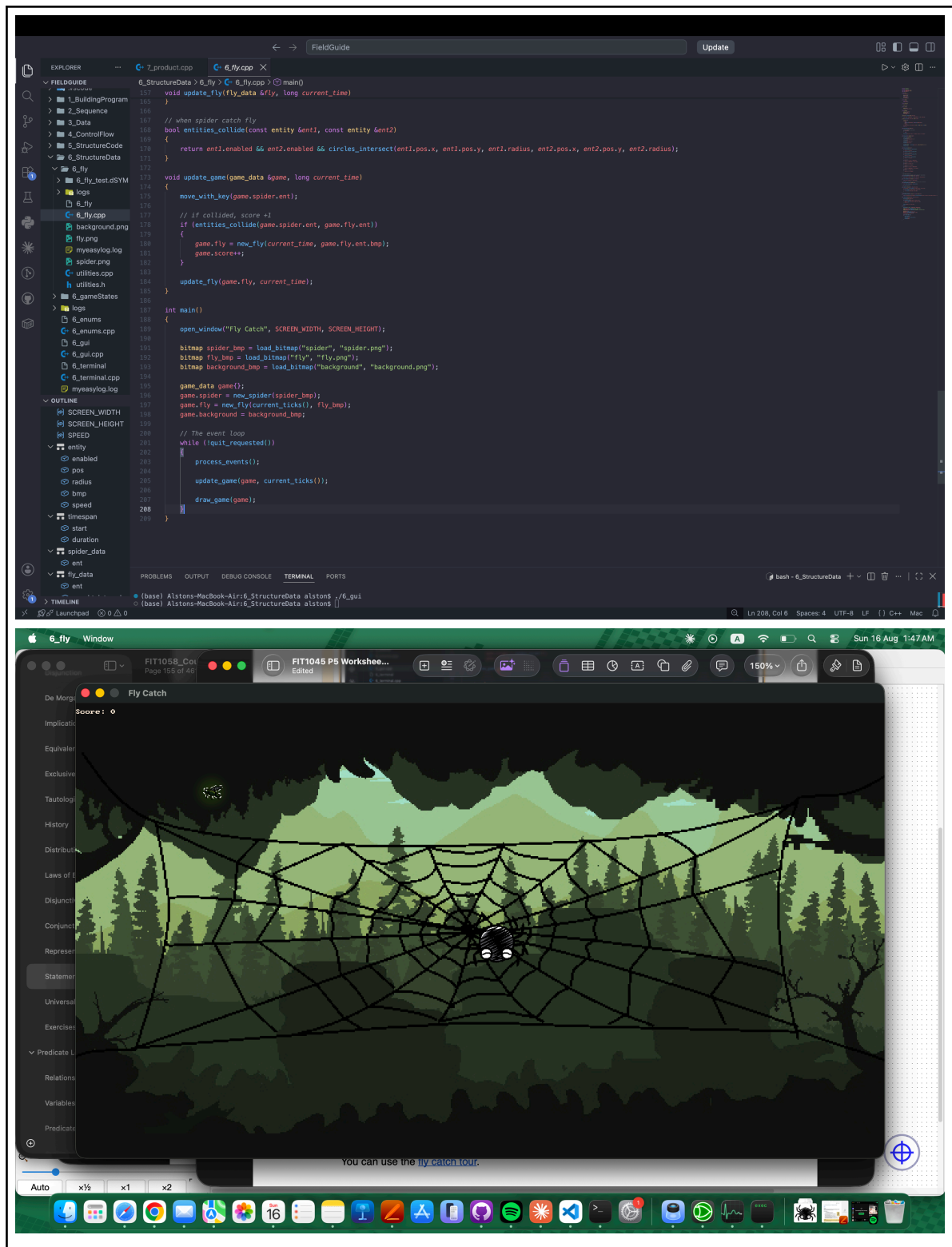
    // check right
    if (ent.pos.x + ent.radius > SCREEN_WIDTH)
    {
        ent.pos.x = SCREEN_WIDTH - 2 * ent.radius;
    }
    // check left
    if (ent.pos.x - ent.radius < 0)
    {
        ent.pos.x = 2 * ent.radius;
    }
    // check down
    if (ent.pos.y + ent.radius > SCREEN_HEIGHT)
    {
        ent.pos.y = SCREEN_HEIGHT - 2 * ent.radius;
    }
    // check up
    if (ent.pos.y - ent.radius < 0)
    {
        ent.pos.y = 2 * ent.radius;
    }

    // Time span function start
    // full within timespan span.start <= time < (span.start + span.duration)
    bool within_timespan(const timespan &span, long time)
    {
        return (time >= span.start) && (time < (span.start + span.duration));
    }

    // when spider catch fly
    bool entities_collide(const entity &ent1, const entity &ent2)
    {
        return ent1.enabled && ent2.enabled && circles_intersect(ent1.pos.x, ent1.pos.y, ent1.radius, ent2.pos.x, ent2.pos.y, ent2.radius);
    }

    void update_game(game_data &game, long current_time)
    {
        move_with_key(game.spider.ent);
        // if collided, score +1
        if (entities_collide(game.spider.ent, game.fly.ent))
        {
            game.fly = new_fly(current_time, game.fly.ent.bmp);
            game.score++;
        }
        update_fly(game.fly, current_time);
    }
}

```



- Complete one of the [Test Your Capabilities](#) tasks from the chapter to see that you can now put these tools together to achieve something yourself.

> I choose Game States.

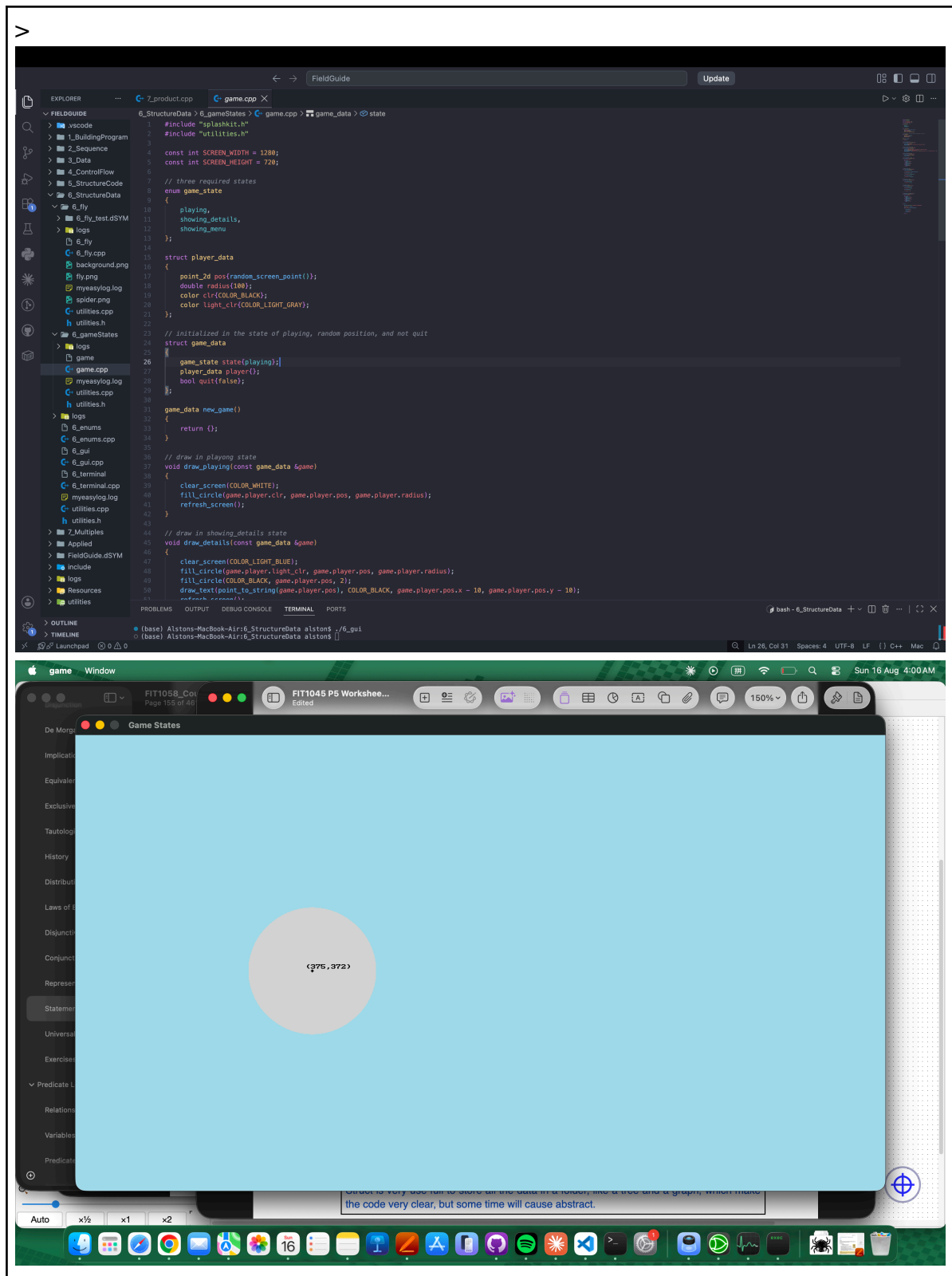
First, I refer to the hint on Field Guide and fly_catch.cpp to have a structure of my program.

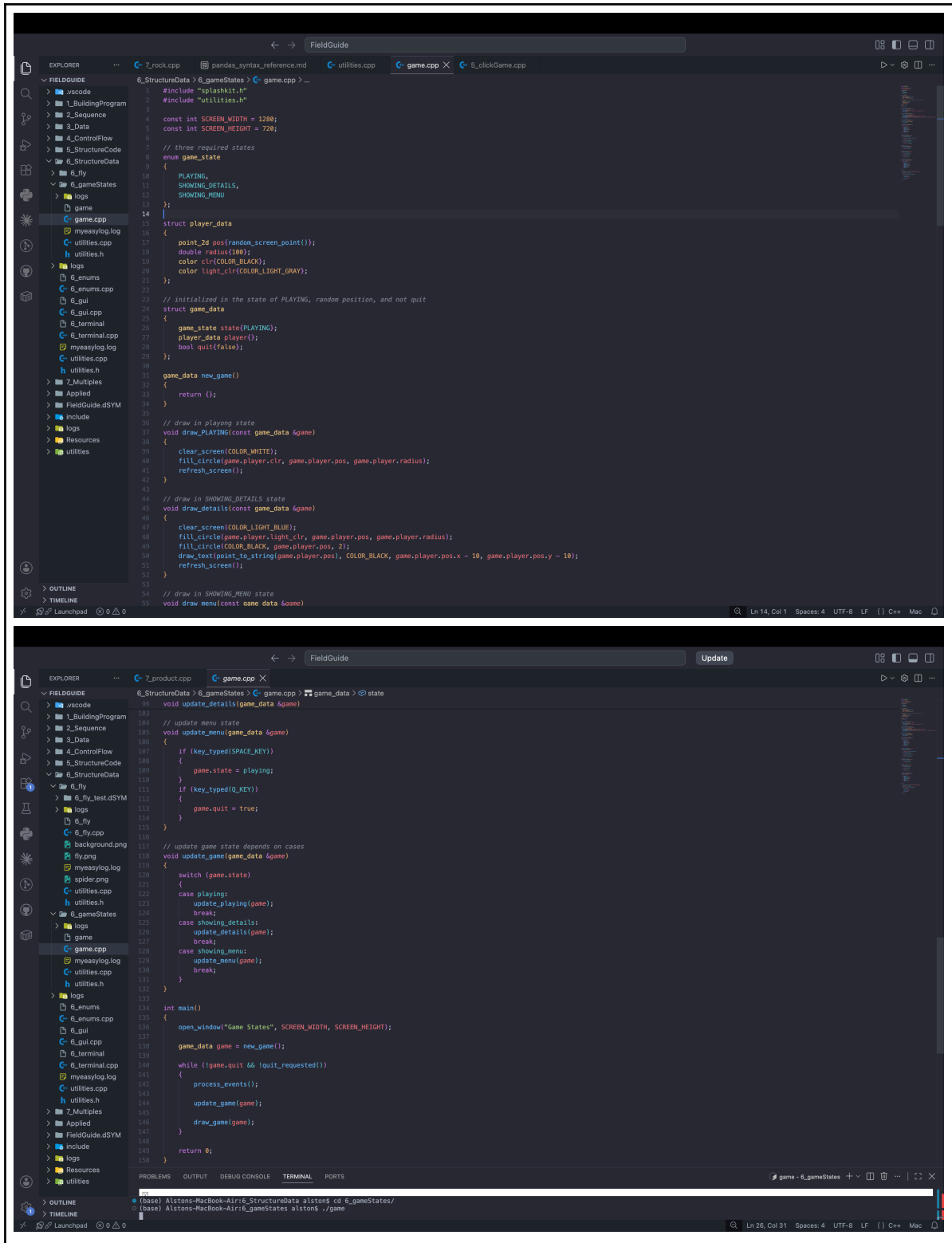
I listed three required game states using enum, use struct game_data to include the game state, player position and quit respond with a initialised with suitable value.

Then I listed all draw and update function of each case and the draw_game() and update_game() function including switch, which is in the hint.

I write each function one by one, from the menu, playing to details.

Each function is very easy to write, just need to follow the instructions, and separate into draw part and update part.





The image displays two screenshots of a Visual Studio Code editor working on a C++ project. The top screenshot shows the 'game.cpp' file with the following code:

```

1 #include "splashkit.h"
2 #include "utilities.h"
3
4 const int SCREEN_WIDTH = 1280;
5 const int SCREEN_HEIGHT = 720;
6
7 // three required states
8 enum game_state
9 {
10     PLAYING,
11     SHOWING_DETAILS,
12     SHOWING_MENU
13 };
14
15 struct player_data
16 {
17     point_2d pos(random_screen_point());
18     double radius(100);
19     color clr(COLOR_BLACK);
20     color light_clr(COLOR_LIGHT_GRAY);
21 };
22
23 // initialized in the state of PLAYING, random position, and not quit
24 struct game_data
25 {
26     game_state state(PLAYING);
27     player_data player();
28     bool quit(false);
29 };
30
31 game_data new_game()
32 {
33     return {};
34 }
35
36 // draw in playing state
37 void draw_PLAYING(const game_data &game)
38 {
39     clear_screen(COLOR_WHITE);
40     fill_circle(game.player.clr, game.player.pos, game.player.radius);
41     refresh_screen();
42 }
43
44 // draw in SHOWING_DETAILS state
45 void draw_DETAILS(const game_data &game)
46 {
47     clear_screen(COLOR_LIGHT_BLUE);
48     fill_circle(game.player.light_clr, game.player.pos, game.player.radius);
49     fill_circle(COLOR_BLACK, game.player.pos, 2);
50     draw_text(point_to_string(game.player.pos), COLOR_BLACK, game.player.pos.x - 10, game.player.pos.y - 10);
51     refresh_screen();
52 }
53
54 // draw in SHOWING_MENU state
55 void draw_MENU(const game_data &game)

```

The bottom screenshot shows the 'game_data.cpp' file with the following code:

```

96 void update_details(game_data &game)
97 {
98     // update menu state
99     void update_menu(game_data &game)
100     {
101         if (key_pressed(SPACE_KEY))
102         {
103             game.state = playing;
104         }
105         if (key_pressed(Q_KEY))
106         {
107             game.quit = true;
108         }
109     }
110
111     // update game state depends on cases
112     void update_game(game_data &game)
113     {
114         switch (game.state)
115         {
116             case playing:
117                 update_playing(game);
118                 break;
119             case showing_details:
120                 update_details(game);
121                 break;
122             case showing_menus:
123                 update_menu(game);
124                 break;
125         }
126     }
127
128 int main()
129 {
130     open_window("Game States", SCREEN_WIDTH, SCREEN_HEIGHT);
131     game_data game = new_game();
132
133     while (!game.quit && !quit_requested())
134     {
135         process_events();
136         update_game(game);
137         draw_game(game);
138     }
139
140     return 0;
141 }

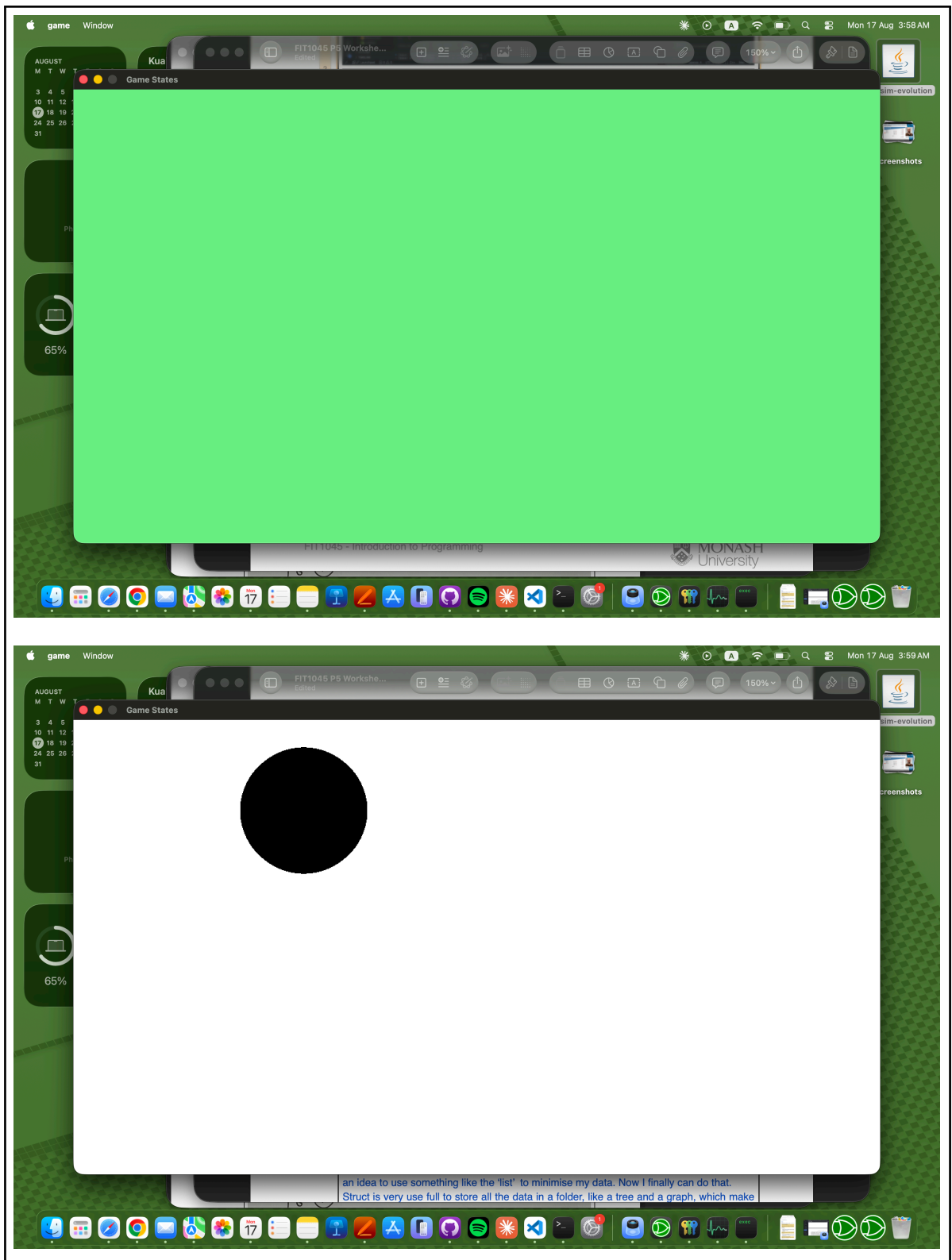
```

The bottom window also shows a terminal output with the following commands:

```

(base) Alstons-MacBook-Air:6_StructureData alstons cd 6_gameStates/
(base) Alstons-MacBook-Air:6_gameStates alstons ./game

```



Tip: Check out naming conventions for your custom types

Make sure to check the [style guide](#) sections around creating fields, structs, and enums.

Reflect:

- What was the most interesting thing you learnt through this task, and why?

> The most interesting thing in this task is about struct, I love it. In P4 click game I have an idea to use something like the 'list' to minimise my data. Now I finally can do that. Struct is very use full to store all the data in a folder, like a tree and a graph, which make the code very clear, but some time will cause abstract.

- How do you see structs helping you create larger programs? Relate this to things you have created during this task.

> I think is very efficient for a larger programs. For example is the "Fly catch" program. This program is very complex and abstract, because is too many link between all the data. But it is useful to structure the code, reduce the thing we need to repeat.

- How are you tracking toward achieving your target grade? How are you going with your planned schedule? Are you keeping your learning plan up to date with adjusted timelines?

> Yes, I think I'm on the road toward my target grade "HD". I spend many time in FIT1045 each week for learning, understanding and improving my programming skill and doing the On Track task. I have some adjust about the timelines but still progressing very fast.

- Looking back, how long did it take to complete this task? What did you spend the most time on?

> I finish this within one day in weekend, spend a lot of time in "Fly catch", "Explore Game States" is too easy compare to it.

Disclose:

- Explain your use of Generative AI for this task. If you didn't use it, just write "I did not use any AI for this task". If you did use it, please state which AI model you used (e.g. ChatGPT) and how.

> I use Claude to learning about the concept of chapter 6. Besides Field Guide, I using [learncpp.com](https://learn.cpp.com) to learn and using Claude to help me for understanding about this task.

Submit:

- Export this worksheet as a PDF.
- Upload your submission to OnTrack:
- Engage with feedback and discussion to get this signed off.